Isaac Simms - Homework 3 - CS425 Fall 2026

Problem 1:
Part i:
At a high level, the round-based synchronous consensus proof is as follows:

1. There are two live processes, p_i has a value of v, and p_j does not.
2. Because p_j does not know about v, p_i must have learned about that value last round. If it had v any earlier, it would have been sent during the last round.
3. Some other process (such as p_k) must have sent v to p_i and crashed before sending v to p_j

The proof breaks down at the second step. That step treats the send of v and its arrival before the timeout as the same event, which can hold only because a round is longer than every message delay. In an asynchronous system p_i can send v and stay healthy while the message is still in transit at the end of the round. In this situation, p_j does not have v at the end of the round with no crashing or failures.

Part ii:
For this part we will use processes p_1, p_2, and p_3 with proposals 1, 2, and 3 respectively. The system includes a failure bound f = 1.

The synchronous algorithm tolerates at most f crashes. Every process runs f+1 rounds in lockstep with each other. If a round lasts longer than the maximum message delay it times out. Process p_i will start with its own proposal. Values_i = v_i. In round r it multicasts only the values it has learned since the previous round. Then it will add whatever proposal arrives before the timeout. After round f+1 it decides the minimum value for each proposal it holds.

Under an asynchronous model p_1's round 1 announcements of proposal 1 to p_2 and p_3 stay in transit until the other two processes have decided on a consensus value. And, round 2 does not send 1 again. Therefore, p_1 is deciding the consensus value of 1 while p_2 and p_3 are deciding on 2. This message delay breaks consensus across the processes, and by extension breaks synchronous methodology.

Problem 2:

Multicast	Value at send	Written at NHX receive	Written at E receive	WRitten at S receive
E1	[0, 1, 0]	[0, 1, 0]	N/A	[0, 1, 0]
N1	[1, 1, 0]	N/A	[1, 1, 0]	[1, 1, 0] (buffered)
S1	[1, 1, 1]	[1, 1, 1]	[1, 1, 1]	N/A
E2	[1, 2, 1]	[1, 2, 1] (buffered)	N/A	[1, 2, 1]
S2	[1, 1, 2]	[1, 1, 2] (buffered)	[1, 1, 2]	N/A
N2	[2, 1, 0]	N/A	[2, 1, 0]	[2, 1, 0]

Problem 3:

Message	Sequence #	NHX behavior	E Behavior	S behavior
E1	1	Delivered at arrival	Delivered at send	Delivered at arrival. Released N1
N1	2	Delivered at send	Delivered at arrival	Held until after E1 arrival
S1	3	Deliver at arrival	Deliver at arrival	Deliver at send
E2	4	Held until after S1 arrival.	Deliver at send	Deliver at arrival
S2	5	Held until S1 arrival. Delivered after the held E2	Deliver at arrival	Held own copy until E2 arrival
N2	6	Held own copy until S1 arrived. Delivered after s2	Deliver on arrival	Held until E2 arrival. Deliver after S2

NHX buffered E2, N2, and S2 so that S1 could be delivered in sequence. E did not buffer anything. S buffered N1 so that E1 could be delivered in sequence and buffered S2 + N2 so that E2 could be delivered in sequence.

Problem 4:

Part i:

C = the set of configurations you can reach from the bivalent apex. C can be reached while the event stays unapplied. An event under this proof is the receipt that process p receives message m (e = p, m). In Lemma 3 the e = (p, m) can sit in buffer, not preventing other receipts and that includes e' = (p, m').

In case II the process has two messages waiting. One of the mesasges is held back in buffer and the system still runs without it being delivered. Every snapshot then on still has the one message waiting in the buffer. At some point the process receives another message. In snapshot path the process accepts the incoming message and in another snapshot it does not. Both of these pathways are legal because the message in the buffer is never manipulated. As long as the only message being held back is the one in the buffer, the system behavior is legal under Lamma 3 in this context.

Part ii:

Again, in this case there are two events waiting at a process and one is held in the buffer. Part ii is asking us to analyze the system under a specific condition and determine if that condition is true/false (legal/not legal). The condition is that given there are more than one event being delivered to this process at the starting snapshot and one of them is going to be held in buffer, is it acceptable to hold back any legal event delivery at that time,

Yes, this is a legal condition. The proof is concerned with other metadata of the event such as which other process it came from or the event's importance. The proof is only concerned with if the event is legal and can happen at that snapshot. If the events in question could be delivered, which in this case both events can, then the proof's logic applies. Delivering both events, delivering either on their own, or keeping both in buffer all still produces a snapshot in which any of those decisions are possible.

Part iii:

Paxos is a way for processes to get closer to consensus by agreeing on one value. It works by iterating over a series of rounds. Each round has a proposer process and a unique "ballot" id for that specific round. It be broken down into three phases:

Election: Proposer node asks what values have been accepted by a group of nodes. If more than one node responds the response with the highest ballot id is accepted.

Bill: The proposer node sends its value for the group of nodes to accept. A value is chosen once the group of nodes accepts it. (usually when more then half have the group accepts the value, it is considered the accepted value)

Law: The proposer node sends out an announcement of the new agreed upon value.

Each round contains each of these steps.

The question asks if an election threshold of N/3+1 and a bill threshold of 2N/3+1 is considered safe. Safe under this context would mean that a later round would be unable to choose a different value, once the accepted value has been determined. This paxos configuration is safe because any election group of that size and any bill group of that size are going to share a process between them. Once the bill group has accepted a value, later rounds will always hear that value from a shared process and propose that same value again.

Problem 6:

An important note here is that this must be a **modification** to the ring based election algorithm as described in lecture. Many of the key factors in that election algorithm must be maintained such as N total processes, one clockwise successor for each process, unique ids, concurrent election protocol starts, and eventual delivery.

Originally, the algorithm elects the process in the ring with the highest identifier. That identifier is carried throughout the election until a different process replaces it with its own identifier because its higher. The process is the leader when it receives its own identifier (because it has the highest identifier). That sparks an elected message to get sent around the ring, each process sending to its clockwise neighbor.

We are tasked with fixing the single leader monopoly issue in this architecture while still satisfying safety and liveness. To accomplish this each election message will carry a token with a set of k identifiers. The election message also carries an identifier which distinguishes it from other election messages. When an election message is received at a process, the process will add its own identifier to the token and then keep only the lowest identifiers, up to the amount of k. That token gets passed around the whole ring. When the election message returns to the original sender, that sending process recognizes that by looking at the message identifier. The sending process stores the content of the token, identifies the lowest process identifier in the token, and sends that out in the elected message. When elected, every process except the initiator stores the set and forwards the message. The initiator does not send it out again so the messages have a stopping point. The processes whose identifiers are in k after this process are the leaders.

This satisfies liveness because the highest initiator is never ignored. The election lap and election lap finish are well defined. This satisfies safety because assuming no crashes, the *elected* variable is only NULL or the k lowest identifiers found by running the algorithm.

Problem 7:

Problem 7 describes a clutter with 5 servers (S1-S5) in it at its third term. S1 is the leader. S1-S3 store log terms [1, 1, 1] while S4-S5 store log terms [2, 2]. The question is whether Raft reaching that snapshot is a violation. If Raft is able to reach that snapshot the sequence of elections, crashes, and/or replications that produce the snapshot is required.

Raft is a consensus algorithm revolving around a log which is replicated across the system. The servers in a cluster agree on the sequence of commands occurring in the log, once those logs are committed. (If the state presented in the S4-S5 logs where committed that would break the completeness of raft since S1, the leader, does not agree on that state)

However, the term 2 entries in the S4-S5 logs are in an uncommitted state and allowed to disagree with other uncommitted states of the system. The snapshot presented in this problem never presents a commit index for those logs. The term number two entities sit at only two of the servers, which is less than the majority. Those entries never reached a majority and were never committed.

The term 1 entries that reside in the logs of S1-S3 are uncommitted as well, as there is no commit index for those entries as well. A leader can commit by replica count only an entry from its current term. Which, the leader is in term three in this example. Since S1 has not appended a term three entry yet, the commit index stays at 0 and there has been no violation of Raft.

Problem 8:

Part i:

In this problem's context hundreds of machines sit on a cord ring. In part i we are asked to build a leader election protocol which picks exactly one leader. That leader must have the second-lowest ID and this needs to be done in very few lookups/messages, it cannot be done through constant chatter.

This can be easily completed through leveraging some of the facts of a chord ring. In a chord ring, every machine already knows the next machine clockwise from it in the ring. That neighboring machine is its successor.

The protocol starts with every machine already storing the ID of its successor as part of chord architecture. IDs only increase as you go clockwise but the **IDs wrap one to form the ring**. Meaning, there is only one machine whose successor has a smaller ID than the machine that sends the initial chord lookup.

Once that machine determines that it is the one with the highest ID it sends one chord lookup with the election payload to the machine with the ID of (*lowest ID successor machine* + 1 mod 2^m). That additional chord message is always going to land on the lowest machine (the successor) which will see that the requested ID sits between itself and its own successor. So, the lowest ID machine forwards that message to the machine with the second lowest ID. That machine accepts the election and becomes the leader.

Part ii:

Safety: To satisfy safety the election must produce exactly one leader at the end. Due to the nature of chord ring architecture, there can only be one node on the ring with the highest ID. That is the only node that is able to send the election message to the second-lowest ID. Meaning, there can only be one leader elected.

Liveness: means that the election must finish. If there are any number of nodes on a chord ring there is always going to be one with the highest ID. From there the rest of the logic is a local comparison the machine already stores so it always completes.

Part iii:

The completion time of this protocol is two hops with a fixed completion cost of O(1). This does not change based on the size of the ring.

The messages:

1. The highest machine sends the routing request for (*lowest ID successor machine* + 1 mod 2^m) to the machine with the lowest ID (its successor).
2. That machine with the lowest ID sees that the request ID sits between itself and its successor, so it forwards it to the machine with the second lowest ID which accepts the election payload.

Part iv:

In this protocol the comparison of IDs is local and does not require the finger table. However, the election itself is a two hop procedure which uses the successors of each node, which are found in the finger tables. If machines have failed and the finger table never got updated that can lead to the route stopping and nobody becoming the leader. Or, the election payload gets routed to a different live machine which does not satisfy the "second lowest ID is the leader" request.

Problem 9:

The architecture in problem 9 is using raft as the election algorithm. There are seven servers in question. A candidate server before the leader after it gets four votes (majority). In traditional Raft, a leader can mark an entry as committed only after that entry has been marked on a majority of servers, so in this case four servers. The internals of the Raft stays the same for problem 9 other than a single change, the commit count. In problem 9 only 3 servers need to store the election entry before the new leader can mark that entry as committed. Everything else stays the same.

The question starts with server A now being the elected leader and log ‘X’ being the entry. X is committed on servers A, B, and C (acceptable under the proposed change to Raft for this problem). Is it possible for a different server than A to get elected to leader without ever having log X touched by the servers which elect the new leader?

The answer is yes. Log X is not on a majority of servers, and a simple majority of servers is what is required to elect a leader in this example. D, E, F, and G could begin an election protocol, choose one of those servers as the candidate leader, and commit a separate log outlining that new server as the leader, all without touching servers A, B, and C.